



МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
ТЕХНОЛОГИЧЕСКОГО ОБРАЗОВАНИЯ**
Кафедра информационных технологий и электронного обучения

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения»
форма обучения – очная

Курсовая работа

«Разработка веб-портфолио студента с использованием современных
фронтенд-технологий»

Обучающегося 4 курса
Гневнова Артема Евгеньевича

Научный руководитель:
кандидат физико-математических наук,
доцент кафедры ИТиЭО
Жуков Николай Николаевич

Санкт-Петербург
2025

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	1
ВВЕДЕНИЕ	2
1 Анализ технологий и проектирование веб-портфолио	4
1.1 Функциональное назначение и структурные особенности веб-портфолио	4
1.2 Анализ современных методов создания фронтенд-приложений	4
1.3 Выбор инструмента для реализации веб-портфолио	6
1.4 Выводы по теоретической части	7
2 Проектирование и программная реализация веб-приложения	9
2.1 Проектирование архитектуры и структуры данных проекта	9
2.2 Разработка логики навигации и программных модулей на Native JS	11
2.3 Реализация адаптивного интерфейса и логики взаимодействия	12
2.4 Публикация и тестирование проекта в среде GitHub Pages	14
2.5 Выводы по практической части	15
ЗАКЛЮЧЕНИЕ	17
ЛИТЕРАТУРА	18

ВВЕДЕНИЕ

Информационные технологии на сегодняшний день являются одной из самых динамично развивающихся отраслей. В условиях высокой конкуренции на рынке труда, требования к молодым специалистам, выпускникам направлений подготовки в области информатики и вычислительной техники, постоянно растут. Для успешного старта карьеры современному студенту недостаточно обладать теоретическими знаниями; необходимо иметь инструмент для наглядной демонстрации практических навыков, реализованных проектов и накопленного за время обучения опыта.

Традиционные способы представления квалификации, такие как бумажное резюме, постепенно уступают место интерактивным решениям. Веб-портфолио становится персональной витриной разработчика, позволяющей не только перечислить достижения, но и продемонстрировать качество кода, владение инструментами верстки и логику построения программных интерфейсов.

Актуальность темы обусловлена современными условиями развития ИТ-индустрии. Профессиональное портфолио становится ключевым инструментом для демонстрации компетенций специалиста. Использование современных фронтенд-технологий, таких как архитектура Single Page Application (SPA) на базе чистого JavaScript, позволяет создать высокопроизводительный и адаптивный продукт, минимизируя зависимость от тяжеловесных внешних библиотек и фреймворков.

Практическая значимость работы заключается в получении масштабируемого программного продукта, который выступает единой точкой доступа к профессиональному профилю автора для презентации навыков при трудоустройстве, а также может служить открытым шаблоном для других обучающихся при создании собственных веб-портфолио на базе аналогичных технологий.

Объект исследования — процесс систематизации, хранения и интерактивного представления образовательных и профессиональных достижений студента в цифровой среде.

Предмет исследования — методы и программные средства фронтенд-разработки (HTML5, CSS3, JavaScript), а также инструменты автоматизации хостинга статических сайтов.

Цель работы — проектирование и программная реализация динамического адаптивного веб-портфолио студента с использованием современных стандартов фронтенд-разработки и механизмов бесшовной навигации.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Изучить требования к структуре и содержанию веб-портфолио программиста.

2. Провести сравнительный анализ инструментов разработки фронтенд-части веб-приложений.
3. Обосновать выбор технологического стека на основе стандартов HTML5, CSS3 и JavaScript без использования внешних зависимостей.
4. Разработать архитектуру и пользовательский интерфейс веб-портфолио.
5. Реализовать функциональные возможности: динамическую навигацию, смену цветовых схем и адаптивную верстку.
6. Осуществить развертывание готового проекта на платформе статического хостинга GitHub Pages.

1 Анализ технологий и проектирование веб-портфолио

1.1 Функциональное назначение и структурные особенности веб-портфолио

Веб-портфолио — это персональный сайт разработчика, предназначенный для демонстрации его проектов, технического стека и профессионального роста. Для студента ИТ-направления такой ресурс служит цифровым доказательством квалификации, дополняя классическое резюме реальными примерами кода.

Анализ успешных примеров реализации персональных ресурсов, таких как портфолио веб-разработчика Кейты Ямады (Keita Yamada) по адресу: <https://keitayamada.com>, показывает, что ключевым фактором успеха является баланс между эстетикой и информационной доступностью [14]. В данном решении акцент сделан на концепцию минимализма, где отсутствие лишних графических элементов и высокая скорость отклика интерфейса позволяют сосредоточить внимание пользователя исключительно на профессиональных навыках автора.

Изучение подобных практик позволяет выделить ряд технических и функциональных требований, которым должно соответствовать эффективное веб-портфолио:

1. Логическая структура. Контент должен быть разбит на понятные блоки: учебные семестры, курсовые проекты, практики и контактная информация. Это позволяет пользователю быстро найти интересующий его раздел.

2. Интеграция с Git-репозиториями. Ключевым требованием является наличие ссылок на исходный код проектов. Для программиста важно показать не только финальный результат (скриншоты или описание), но и саму архитектуру кода, культуру именования переменных и логику решения задач.

3. Адаптивная верстка. Интерфейс должен корректно работать на устройствах с любым разрешением экрана. Это не только вопрос удобства, но и показатель качества владения фронтенд-технологиями [16].

4. Производительность и скорость загрузки. Чистый код и отсутствие лишних зависимостей напрямую влияют на скорость работы сайта. Быстрый отклик интерфейса создает положительное впечатление о навыках оптимизации у автора.

Подытоживая, портфолио — это полноценный программный продукт, который должен быть удобным, быстрым и прозрачным с точки зрения реализации.

1.2 Анализ современных методов создания фронтенд-приложений

Современная веб-разработка представляет собой многослойный процесс, где выбор инструментов напрямую определяет производительность и масштабируемость итогового продукта. Основу клиентской части любого веб-ресурса (фронтенда) составляют три ключевых стандарта:

1. HTML5 (HyperText Markup Language). Данная технология отвечает за логическую разметку документа. Современный стандарт HTML5 предлагает набор семантических элементов (таких как `<nav>`, `<main>`, `<section>`, `<footer>`), которые позволяют поисковым системам и браузерам корректно интерпретировать структуру страницы [9]. Использование семантики критически важно для доступности интерфейса (accessibility), так как это позволяет экранным дикторам корректно озвучивать контент.

2. CSS3 (Cascading Style Sheets). Язык описания внешнего вида. Современный CSS3 включает в себя мощные инструменты верстки, такие как Flexbox и CSS Grid, позволяющие строить адаптивные сетки без сторонних библиотек [3]. Особое значение имеют CSS-переменные (Custom Properties), которые позволяют динамически менять параметры (например, палитру цветов) во всем приложении, что необходимо для реализации тем оформления.

3. JavaScript (ECMAScript 6+). Язык сценариев, превращающий статичную разметку в интерактивное приложение. Стандарт ES6+ привнес возможности, которые ранее были доступны только в сторонних библиотеках: модульность, классы и асинхронность [1]. С помощью JS разработчик напрямую манипулирует DOM-деревом (Document Object Model) — интерфейсом, позволяющим изменять содержимое страницы в процессе работы пользователя [15].

Помимо программирования «с нуля» на базовых технологиях, в индустрии широко применяются вспомогательные инструменты и готовые платформы:

- Фронтенд-фреймворки (React, Vue.js, Angular). Они автоматизируют управление интерфейсом. React использует Virtual DOM для быстрой отрисовки изменений, Vue.js предлагает удобную структуру файлов (SFC), а Angular — строгую архитектуру на базе TypeScript. Эти инструменты эффективны в больших командах, но требуют сложной настройки и увеличивают «вес» итогового кода.

- Системы управления контентом (CMS) и No-code конструкторы (Tilda, Lektor, Wix, WordPress). Эти инструменты позволяют собирать веб-страницы через визуальные редакторы без написания кода. Несмотря на их популярность для бизнес-задач и быстрого запуска лендингов, они существенно ограничивают возможности разработчика в оптимизации производительности. Главным недостатком подобных систем для программиста является невозможность продемонстрировать индивидуальные архитектурные решения и навыки написания чистого кода.

Важным этапом эволюции веба стала архитектура Single Page Application (SPA) [12]. Традиционная модель (MPA) подразумевает, что при переходе по ссылке браузер запрашивает новый HTML-файл у сервера и перерисовывает страницу с нуля. Это ведет к «миганию» экрана и потере состояния (сбросу прокрутки или анимаций). В SPA сайт загружается один раз, а навигация происходит на стороне клиента: JavaScript перехватывает клик, предотвращает перезагрузку и мгновенно заменяет контент в основном блоке [2]. Вследствие этого интерфейс работает плавно, как мобильное приложение.

Можно сделать вывод, что наличие такого разнообразия подходов ставит перед разработчиком задачу выбора наиболее эффективного набора технологий. Необходимо определить, будет ли использование мощных фреймворков или готовых конструкторов оправдано для конкретной задачи, или стоит отдать предпочтение стандартным веб-инструментам (Native JS) для достижения максимальной легкости и прозрачности приложения.

1.3 Выбор инструмента для реализации веб-портфолио

Для реализации веб-портфолио можно выделить три основных подхода, каждый из которых обладает своими преимуществами и недостатками в зависимости от конечных целей проекта и требуемого уровня контроля над кодом.

1. Использование No-code конструкторов и CMS (Tilda, Wix, WordPress). Данный метод позволяет получить готовый результат в кратчайшие сроки, однако он полностью скрывает внутреннюю логику приложения. Для студента-разработчика этот вариант является наименее предпочтительным, так как он не позволяет продемонстрировать владение технологиями и культуру написания кода. Более того, сайты на базе таких платформ часто имеют избыточный вес и низкие показатели производительности из-за большого количества системных скриптов, что критично для современных веб-стандартов.

2. Разработка на базе фронтенд-фреймворков (React, Angular, Vue.js). Этот путь является индустриальным стандартом для создания сложных систем со множеством динамических данных и большой командой разработчиков. Однако для персонального портфолио использование данных инструментов часто оказывается избыточным. Применение React или Angular требует настройки сложной среды окружения, использования сборщиков и подключения объемных библиотек, вес которых может значительно превышать объем полезного контента сайта. Это создает неоправданную нагрузку на клиентское устройство и усложняет дальнейшую поддержку проекта.

3. Разработка с использованием Native JavaScript (Vanilla JS) и архитектуры SPA. Данный подход предполагает использование стандартных возможностей веб-платформы

без привлечения тяжеловесных внешних зависимостей, что является оптимальным решением для текущей курсовой работы. Отказ от сторонних фреймворков обеспечивает полную прозрачность архитектуры, так как каждая функция и программный модуль реализуются автором вручную, гарантируя глубокое понимание всех процессов внутри приложения. Отсутствие избыточного кода напрямую влияет на достижение максимальной производительности, обеспечивая мгновенную загрузку страниц и высокую скорость рендеринга интерфейса [6]. Кроме того, самостоятельная реализация таких механизмов, как маршрутизация через History API и динамическое управление DOM-деревом, позволяет наглядно продемонстрировать фундаментальные навыки программирования и владение актуальными веб-стандартами [13].

Для наглядности сопоставим рассмотренные методы по ключевым критериям:

Таблица 1 — Сравнительный анализ инструментов разработки

Критерий сравнения	No-code/CMS	Фреймворки (React/Vue)	Native JS (SPA)
Контроль над кодом	Минимальный	Высокий	Полный
Производительность	Низкая	Средняя	Высокая
Сложность поддержки	Низкая	Высокая	Средняя
Демонстрация навыков	Нет	Да	Да

В итоге, на основании проведенного анализа в таблице 1, для разработки был выбран стек технологий на базе HTML5, CSS3 и Native JavaScript с точечной интеграцией внешних статических ресурсов. Такой подход позволяет создать производительное и современное приложение, которое эффективно решает задачу презентации профессиональных навыков, сохраняя при этом чистоту и прозрачность программной реализации [7].

1.4 Выводы по теоретической части

В рамках теоретической части были выполнены задачи, направленные на обоснование структуры и выбор инструментов реализации проекта. По итогам главы получены следующие результаты:

1. Изучены требования к структуре и содержанию веб-портфолио. Установлено, что для эффективной демонстрации квалификации программиста ресурс должен включать в себя четкую иерархию (разделение по семестрам, курсовым работам и практикам), адаптивную верстку и обязательную интеграцию с репозиториями в Git-системах.

2. Проведен сравнительный анализ инструментов разработки фронтенд-части. В ходе сопоставления визуальных конструкторов, тяжеловесных фреймворков (React, Angular) и нативных веб-технологий были выявлены преимущества и недостатки каждого подхода. Установлено, что использование сторонних фреймворков для персонального сайта часто ведет к избыточности кода и снижению производительности.

3. Обоснован выбор технологического стека. Вследствие проведенного анализа было принято решение о разработке приложения на базе стандартов HTML5, CSS3 и JavaScript (ES6+) без использования внешних программных зависимостей. Данный выбор позволяет минимизировать вес приложения, обеспечить его мгновенную загрузку и продемонстрировать глубокое владение базовыми механизмами веб-платформы, такими как манипуляция DOM-деревом и управление состоянием через History API.

2 Проектирование и программная реализация веб-приложения

2.1 Проектирование архитектуры и структуры данных проекта

Процесс создания веб-портфолио был организован в соответствии с итерационной моделью разработки. Выбор данной методологии обусловлен возможностью последовательного наращивания функционала: от проектирования структуры данных до реализации интерактивных модулей, с возможностью тестирования и отладки каждого этапа в отдельности [4].

Методология управления задачами включала в себя следующие этапы:

1. Проектирование и подготовка данных. Определение формата хранения информации о проектах. Было принято решение использовать структуру JavaScript-объектов вместо статического наполнения HTML-страниц, что позволило гибко управлять данными о семи семестрах обучения, курсовых работах и практиках.

2. Разработка интерфейса и верстка. Создание визуального каркаса приложения с использованием семантических тегов HTML5 и каскадных таблиц стилей CSS3. На этом этапе особое внимание уделялось разработке адаптивной сетки, обеспечивающей корректное отображение контента на устройствах с различным разрешением экрана [8].

3. Программная реализация логики. Разработка ядра системы на «чистом» JavaScript (Native JS), включая механизмы навигации без перезагрузки страниц (SPA), управление состоянием через History API и логику переключения цветовых схем.

4. Отладка и развертывание. Финальная проверка работоспособности всех модулей, кроссбраузерное тестирование и публикация проекта в сети Интернет.

Инструментальное обеспечение разработки: в качестве основной среды разработки (IDE) использовался редактор Visual Studio Code. Выбор данного инструмента обоснован наличием встроенных средств отладки и расширений для удобной навигации по файловой структуре проекта. Внешний вид рабочей среды с открытыми файлами проекта представлен на рисунке 1.

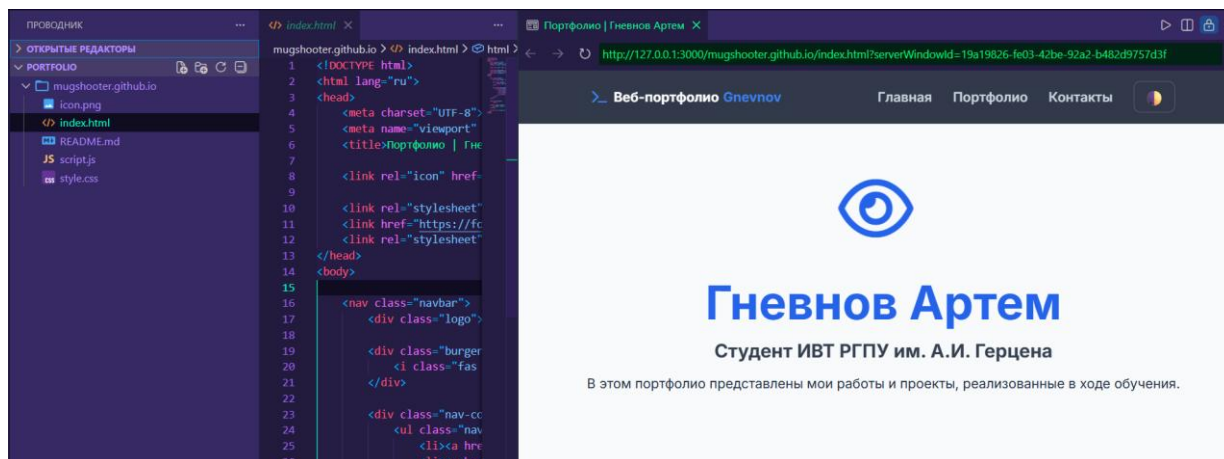


Рисунок 1 — Интерфейс среды разработки Visual Studio Code с открытыми файлами проекта.

Для контроля версий и управления изменениями применялась система Git, а в качестве удаленного репозитория — платформа GitHub.

Ключевым инструментом отладки программной логики и визуального отображения выступили инструменты разработчика браузера (Chrome DevTools). С их помощью осуществлялся мониторинг состояния DOM-дерева, отслеживание ошибок в консоли JavaScript, а также проводилось тестирование адаптивности интерфейса [5]. Функция эмуляции мобильных устройств в Chrome DevTools позволила убедиться в корректности работы бургер-меню и перестроении карточек проектов на малых экранах. Пример процесса тестирования мобильной версии сайта с помощью данного инструмента показан на рисунке 2.

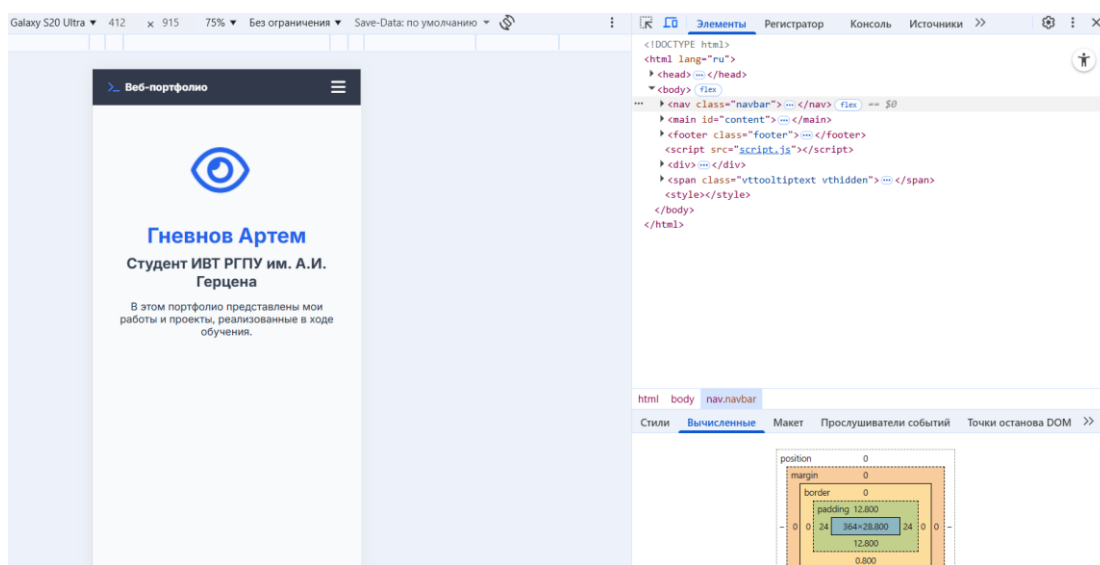


Рисунок 2 — Тестирование адаптивности интерфейса в режиме Device Mode (Chrome DevTools).

2.2 Разработка логики навигации и программных модулей на Native JS

Архитектура разработанного веб-портфолио основана на принципе разделения контента, оформления и логики [4]. Особенностью проекта является использование модели Single Page Application (SPA), где всё приложение функционирует в рамках одной HTML-страницы, а смена разделов происходит динамически без перезагрузки браузера.

Взаимодействие компонентов системы: Функционирование приложения обеспечивается за счет интеграции трех основных файлов. Связи между компонентами прописаны в основном файле разметки index.html. В заголовке документа (<head>) подключаются внешние шрифты и каскадные таблицы стилей:

```
<link rel="stylesheet" href="style.css">  
<link rel="stylesheet" href="https://fonts.googleapis.com/...family=Inter">  
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/.../all.min.css">
```

Управляющий сценарий script.js подключается в самом конце тела документа перед закрывающим тегом </body>. Такое расположение гарантирует, что скрипт начнет исполнение только после полной загрузки DOM-дерева, что критически важно для корректной манипуляции элементами страницы.

Взаимосвязь файлов организована следующим образом:

1. index.html — выступает в роли единой входной точки (Entry Point). Он содержит каркас навигационной панели, подвал и пустой контейнер <main id="content">, который служит целевым объектом для динамического внедрения контента.

2. style.css — единый центр визуального управления. Реализация стилей базируется на использовании CSS-переменных (Custom Properties), определенных в псевдоклассе :root. Это позволяет централизованно управлять цветовой палитрой и реализовать мгновенную смену тем оформления.

3. script.js — программное ядро, выполняющее роль контроллера. Оно управляет маршрутизацией, обрабатывает события пользователя и генерирует интерфейс на основе встроженных структур данных.

Организация и структурирование данных: для обеспечения масштабируемости проекта вся информация об учебной деятельности была вынесена в константы JavaScript. Данные структурированы в виде иерархических массивов объектов. Такой подход позволяет обновлять контент, не затрагивая верстку и логику работы приложения.

Примером организации данных служит массив semesters, где каждый элемент представляет собой объект с уникальным идентификатором, заголовком и вложенным списком предметов. Фрагмент реализации хранилища данных в программном коде представлен на рисунке 3.

```

JS script.js
mugshooter.github.io > JS script.js > pages > portfolio
1  const semesters = [
2    { id: 1, title: "1 Семестр", subjects: [
3      {name: "Дискретная математика", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Дискретная%20математика%201"},
4      {name: "Информатика", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Информатика"},
5      {name: "Основы комп. алгебры", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Основы%20компьютерной%20алгебры"},
6      {name: "Лин. алгебра и теория матриц", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Линейная%20алгебра%20и%20теория%20матриц"},
7      {name: "ИТ в математике", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Информационные%20технологии%20в%20математике"},
8      {name: "ИТ в физике", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Информационные%20технологии%20в%20физике"},
9      {name: "Физика", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Физика"},
10     {name: "Философия", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Философия"},
11     {name: "БЖД", link: "https://github.com/mugshooter/repository/tree/main/1%20SEM/Безопасность%20жизнедеятельности"}
12   ]},

```

Рисунок 3 — Реализация хранилища данных в виде массивов объектов в файле script.js.

Использование подобных структур данных позволяет реализовать универсальные функции рендеринга. Вместо дублирования HTML-кода для каждой страницы, приложение программно обходит массивы и автоматически формирует карточки проектов. Это минимизирует объем кода, исключает ошибки при ручном наполнении страниц и значительно упрощает техническую поддержку портфолио в будущем.

2.3 Реализация адаптивного интерфейса и логики взаимодействия

Основная логика веб-приложения сосредоточена в файле script.js. Ключевой задачей программирования была реализация бесшовного пользовательского опыта (UX) на базе архитектуры Single Page Application (SPA) [1], что позволяет переключаться между разделами без перезагрузки страницы.

Механизм динамической навигации. Центральным элементом системы является функция `navigate(pageId, updateHistory = true, extra = null)`, программный код которой представлен на рисунке 4. При вызове функции происходит обновление содержимого контейнера `contentDiv.innerHTML` путем внедрения, соответствующего HTML-шаблона из объекта `pages` [10].

Особенностью реализации является ветвление логики:

- Если параметр `pageId` равен «portfolio», функция запускает процесс рендеринга данных через `renderPortfolio()`.
- В случае передачи аргумента `extra` (идентификатор семестра), система находит нужный объект в массиве `semesters` и отображает список дисциплин через вызов `showSubjects(sem, false)`.

Завершающим этапом работы функции является программная установка задержки анимации (`animationDelay`) для каждой карточки через метод `querySelectorAll('.card')`, что создает эффект плавного каскадного появления элементов.

```

137 function navigate(pageId, updateHistory = true, extra = null) {
138     contentDiv.innerHTML = pages[pageId] || pages['home'];
139
140     if (pageId === 'portfolio') {
141         renderPortfolio();
142         if (extra) {
143             const sem = semesters.find(s => s.id == extra);
144             if (sem) showSubjects(sem, false);
145         }
146     }
147
148     if (updateHistory) {
149         const url = new URL(window.location);
150         url.searchParams.set('page', pageId);
151         if (extra) url.searchParams.set('sem', extra);
152         else url.searchParams.delete('sem');
153
154         window.history.pushState({ pageId, semId: extra }, '', url);
155     }
156
157     const cards = document.querySelectorAll('.card');
158     cards.forEach((card, i) => card.style.animationDelay = `${i * 0.05}s`);
159 }

```

Рисунок 4 — Программный код функции navigate, управляющей состоянием контента.

Реализация клиентского роутинга. Для обеспечения корректной работы истории браузера внутри функции navigate реализовано взаимодействие с History API. С помощью объекта URL и метода window.history.pushState() приложение динамически обновляет адресную строку, добавляя параметры page и sem. Это позволяет пользователю сохранять прямые ссылки на конкретные разделы.

Для того чтобы приложение реагировало на стандартные кнопки браузера «Назад» и «Вперед», добавлен слушатель события popstate. Он извлекает параметры из текущего URL через URLSearchParams и вызывает навигацию с параметром updateHistory = false, предотвращая заикливание истории. Пример формирования динамического URL-адреса с параметрами семестра показан на рисунке 5.

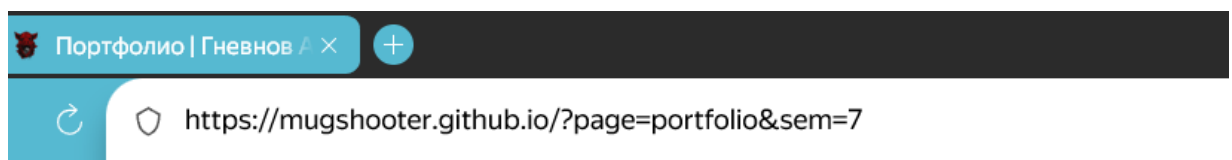


Рисунок 5 — Формирование динамического URL-адреса с параметрами семестра в адресной строке.

Дополнительные функциональные возможности. В проекте реализованы модули, отвечающие за интерактивность и адаптивность интерфейса:

1. Управление цветовой схемой. Логика переключения тем реализована через обработчик `themeBtn.onclick`. Скрипт проверяет текущее значение атрибута `data-theme` у `document.documentElement` и инвертирует его (`light/dark`), что мгновенно меняет значения всех CSS-переменных.

2. Адаптивное меню. Для мобильных устройств реализована логика «бургер-меню». Скрипт отслеживает клик по иконке `#burger`, переключая класс `.active` у контейнера навигации и меняя визуальное состояние иконки с «fa-bars» на «fa-times».

Демонстрация работы темной темы и активного мобильного меню представлена на рисунке 6.

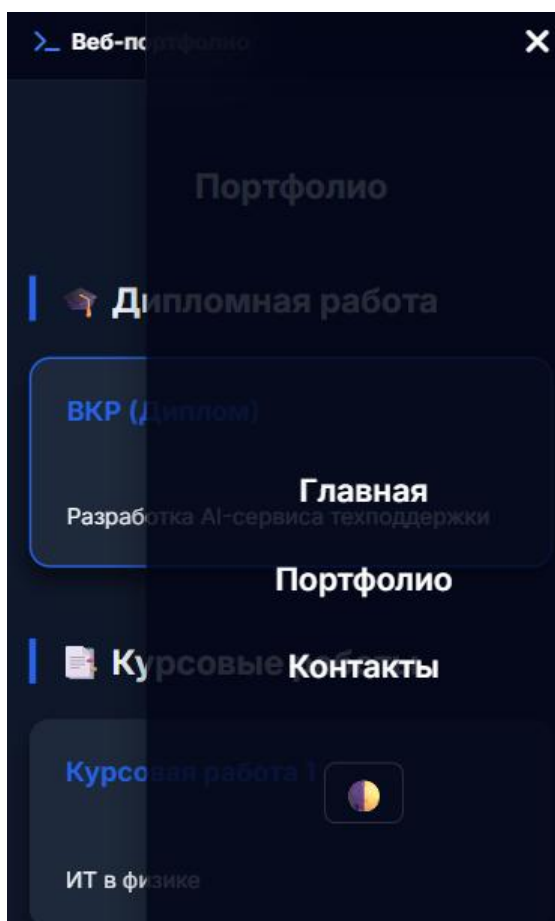


Рисунок 6 — Интерфейс приложения в темной теме оформления и мобильном режиме (бургер-меню).

2.4 Публикация и тестирование проекта в среде GitHub Pages

Финальным этапом разработки стала публикация веб-портфолио в открытом доступе. Для хостинга был выбран сервис GitHub Pages, так как он оптимизирован для размещения статических сайтов и предоставляет бесплатный SSL-сертификат [11].

Процесс деплоя и конфигурации включал следующие этапы:

1. Подготовка репозитория. Все исходные файлы `index.html`, `style.css`, `script.js` и ассеты были загружены в публичный репозиторий на GitHub.

2. Настройка GitHub Actions. В настройках репозитория раздела Pages была активирована автоматическая сборка. При каждой загрузке изменений в ветку `main` запускается встроенный процесс `workflow`, который проверяет целостность файлов и обновляет содержимое сайта.

3. Конфигурация домена. После завершения процесса сборки (GitHub Actions) проекту был присвоен доменный адрес.

Статус успешной публикации и итоговый URL-адрес проекта представлены на рисунке 7.

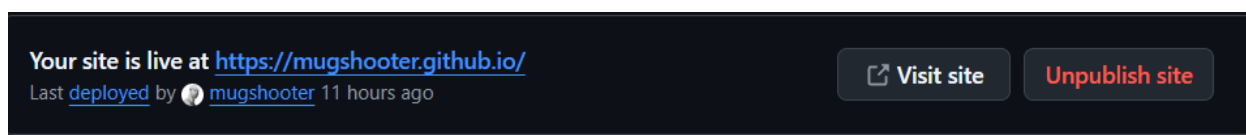


Рисунок 7 — Панель управления GitHub Pages со статусом «Your site is live».

Развертывание на реальном сервере позволило провести финальное тестирование. Была проверена скорость отрисовки карточек, а также работа History API при прямых заходах по ссылкам. Ознакомиться с итоговым результатом работы веб-портфолио можно в открытом доступе по адресу: <https://mugshooter.github.io/>. Исходный код проекта и история разработки доступны в публичном репозитории GitHub: <https://github.com/mugshooter/mugshooter.github.io>

2.5 Выводы по практической части

В ходе разработки был пройден полный цикл создания программного продукта: от проектирования логической структуры данных до финального тестирования и размещения в открытом доступе. На основании проведенных работ можно сделать следующие выводы в соответствии с поставленными задачами:

1. Разработана архитектура и пользовательский интерфейс веб-портфолио. Создана модульная структура приложения на базе технологий HTML5 и CSS3. Использование семантической разметки и разделение контента на логические блоки позволило сформировать интуитивно понятный интерфейс, отвечающий современным стандартам веб-дизайна.

2. Реализованы ключевые функциональные возможности. С помощью «чистого» JavaScript (Native JS) разработана система динамической навигации в формате SPA,

исключающая перезагрузку страниц. Интегрирован механизм смены цветовых схем (светлая/темная тема) и обеспечена полная адаптивность интерфейса для корректной работы на различных типах устройств.

3. Осуществлено развертывание проекта на GitHub Pages. Готовое решение успешно опубликовано на платформе статического хостинга. В процессе деплоя была проведена проверка работоспособности роутинга и корректности путей к ресурсам в условиях реальной сети.

Результатом главы стало полностью функциональное и доступное в сети веб-портфолио, решающее задачу систематизации учебных достижений за весь период обучения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была выполнена разработка и программная реализация персонального веб-портфолио методами современных фронтенд-технологий.

В теоретической части были проанализированы ключевые требования к структуре цифрового портфолио программиста и проведен сравнительный анализ различных подходов к его созданию. В результате исследования был обоснован выбор архитектуры Single Page Application (SPA) на базе «чистого» JavaScript. Такой подход позволил создать производительный продукт с полной прозрачностью программного кода, исключив зависимость от громоздких сторонних библиотек.

В практической части была осуществлена программная разработка приложения с использованием стека технологий HTML5, CSS3 и JavaScript стандарта ES6+. В отличие от использования готовых генераторов сайтов или CMS, в данной работе была самостоятельно реализована система динамической навигации и клиентского роутинга через History API. Также был разработан программный механизм управления состоянием интерфейса для переключения цветовых схем и внедрена адаптивная сетка, гарантирующая корректное отображение контента на любых типах устройств. Завершающим этапом стала настройка автоматизированного процесса публикации проекта в облачную среду через сервис GitHub Pages.

Таким образом, цель курсовой работы была достигнута, а все задачи выполнены в полном объеме. Итоговый программный продукт демонстрирует результаты учебной деятельности за весь период обучения. Созданное портфолио готово к использованию в качестве персонального профиля при трудоустройстве и имеет высокий потенциал для дальнейшего функционального расширения.

ЛИТЕРАТУРА

1. Флэнаган Д. JavaScript. Полное руководство, 7-е изд. — Москва: Вильямс, 2021. — 720 с.: ил. — ISBN 978-5-907203-79-2.
2. Миковски М., Пауэлл Д. Разработка одностраничных веб-приложений: ДМК Пресс, 2023. — 514 с.: ил. — ISBN 978-5-89818-353-0.
3. Янцев, В. В. Разработка web-страниц на HTML, CSS и JavaScript : учебное пособие для СПО / В. В. Янцев. — 2-е изд., стер. — Санкт-Петербург: Лань, 2025. — 148 с. — ISBN 978-5-507-52427-3.
4. Ермаков, С. Р. Веб-разработка: учебно-методическое пособие / С. Р. Ермаков. — Москва: РТУ МИРЭА, 2025. — 95 с. — ISBN 978-5-7339-2593-6.
5. Аграновский, А. В. Тестирование веб-приложений: учебное пособие / А. В. Аграновский. — Санкт-Петербург: ГУАП, 2020. — 155 с. — ISBN 978-5-8088-1515-5.
6. Ollila R., Mäkitalo N., Mikkonen T. Modern Web Frameworks: A Comparison of Rendering Performance: IEEE: Journal of Web Engineering, 2022. — Том 21, Вып. 3. — С. 789-813. — DOI: 10.13052/jwe1540-9589.21312.
7. Sharma A., Pandey Y. Design and implementation of fitness management website: IEEE: 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N), 2022. — С. 1380-1384. — DOI: 10.1109/ICAC3N56670.2022.10074213.
8. Frain B. Responsive Web Design with HTML5 and CSS: Build future-proof responsive websites using the latest HTML5 and CSS techniques: IEEE: Packt Publishing, 2025. — 428 с. — ISBN 978-1-83702-822-1.
9. «HTML Living Standard» HTML Standard: сайт. — 2025. — URL: <https://html.spec.whatwg.org/multipage> (дата обращения: 18.12.2025).
10. «JavaScript» JavaScript | MDN Web Docs: сайт. — 2025. — URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 18.12.2025).
11. «Документация по GitHub Pages» GitHub Docs: сайт. — 2025. — URL: <https://docs.github.com/ru/pages> (дата обращения: 21.12.2025).
12. «Гайд по микрофронтендам на single-spa» Habr: сайт. — 2025. — URL: https://habr.com/ru/companies/ecom_tech/articles/766978 (дата обращения: 22.12.2025).
13. «Роутинг и рендеринг страниц на стороне клиента с помощью History API и динамического импорта» Habr: сайт. — 2025. — URL: <https://habr.com/ru/articles/530746> (дата обращения: 22.12.2025).
14. «Personal Portfolio Keita Yamada» Keita Yamada Website: сайт. — 2025. — URL: <https://keitayamada.com> (дата обращения: 24.12.2025).

15. «Понимание DOM-дерева» DigitalOcean Community: сайт. — 2025. — URL: <https://www.digitalocean.com/community/tutorial-series/understanding-the-dom> (дата обращения: 24.12.2025).

16. «Основы адаптивного веб-дизайна» Google Web Fundamentals: сайт. — 2025. — URL: <https://web.dev/responsive-web-design-basics> (дата обращения: 24.12.2025).